A Reproducible Word2Vec Framework with CBOW and Skip-gram in PyTorch

Zeeshan Ahmad
Registration No: 70169515
University of Lahore, Sargodha Campus
Sargodha, Pakistan
70169515@student.uol.edu.pk

***Abstract*—This paper presents a reproducible implementation and empirical analysis of Word2Vec using Continuous Bag-of-Words and Skip-gram architectures in PyTorch. The proposed framework covers corpus preprocessing, vocabulary construction, dataset generation, model training, checkpoint management, and intrinsic evaluation using nearest-neighbor and analogy tasks. To improve statistical reliability, experiments are conducted across three random seeds, and ablation studies are performed over context window size and embedding dimensionality. On the study corpus, Skip-gram achieves a lower mean best validation loss (3.5006) than CBOW (3.6948), indicating superior representation quality under the same optimization setting. Ablation results further show that larger context windows and higher embedding dimensions improve validation performance for this setup. In addition to model outputs, the framework generates structured logs, figures, and tabular summaries that support transparent reporting and experiment traceability. The findings validate the effectiveness of the proposed pipeline for controlled Word2Vec experimentation and provide a strong baseline for future extensions such as negative sampling and large-scale corpus training.**

***Index Terms*—Word2Vec, CBOW, Skip-gram, word embeddings, PyTorch, reproducibility**

I. Introduction

Distributed word representations are a foundational component in natural language processing because they capture semantic and syntactic regularities in continuous vector spaces [1], [4], [6]. Word2Vec popularized two efficient neural objectives, CBOW and Skip-gram, for learning such representations from raw text [5], [6]. These embeddings remain widely used in low-resource systems and as conceptual building blocks for larger language modeling pipelines.

This paper reports an end-to-end Word2Vec framework in PyTorch with experiment management aimed at reproducible empirical analysis. The implementation includes data preprocessing, training, checkpointing, evaluation, and artifact generation (figures, tables, and logs). The main contributions are:

- A clean implementation of CBOW and Skip-gram with reproducible training settings.
- A comparative empirical study over three seeds for both models.
- Ablation experiments on context window and embedding dimensionality.
- Automatic generation of paper-ready artifacts for reporting and reproducibility.

II. Related Work

Neural language modeling traces back to distributed representation learning and probabilistic neural language models [1], [4], [9]. Later work expanded neural NLP with unified and task-general representation frameworks [2], [3]. Continuous-space language modeling and scalable training methods further improved practicality for larger corpora [7], [8], [10].

Word2Vec introduced computationally efficient objectives and demonstrated linguistic regularities through vector arithmetic [5], [6]. The present study follows the CBOW and Skip-gram formulation while emphasizing reproducible experimentation and structured evaluation.

III. Methodology

A. Preprocessing and Vocabulary

The corpus is lowercased and tokenized, then converted into indexed sequences via a vocabulary containing special tokens for padding and unknown words. Vocabulary filtering by minimum frequency is supported. For this study, min-count is set to 1 to retain relational terms required for intrinsic semantic evaluation.

B. CBOW and Skip-gram Objectives

Given context words around position t, CBOW predicts the center word by maximizing:

$$\log p(w_t \mid w_{t-c}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+c}) \tag{1}$$

Skip-gram predicts surrounding words from the center word by maximizing:

$$\sum_{-c \le j \le c, j \ne 0} \log p(w_{t+j} \mid w_t) \tag{2}$$

Both models are trained with full softmax and cross-entropy loss for conceptual simplicity.

C. Training and Evaluation

Training uses stochastic gradient descent with fixed learning rate. For each run, best and last checkpoints are saved. Evaluation supports nearest-neighbor retrieval and analogy completion using cosine similarity on normalized embedding vectors.

IV. Experimental Setup

A. Dataset Statistics

The working corpus contains 29 sentences, 274 total tokens, and 41 unique tokens. With min-count 1, the resulting vocabulary size is 43; with min-count 3, it is 24.

B. Main Configuration

Main experiments use embedding dimension 100, window size 2, min-count 1, batch size 256, 5 epochs, and learning rate 0.05. Each model is trained with seeds 42, 7, and 123.

C. Ablation Configuration

Window-size ablation: $w \in \{1, 2, 4\}$ using embedding dimension 100. Embedding-size ablation: $d \in \{50, 100, 200\}$ using window size 2.

V. Results and Discussion

A. CBOW vs Skip-gram

Table I summarizes the three-seed comparison. Skip-gram outperforms CBOW in mean best validation loss.

TABLE I
Main Results Across Three Seeds

Model	Runs	Mean Best Val Loss	Std
CBOW	3	3.6948	0.0549
Skip-gram	3	3.5006	0.1163

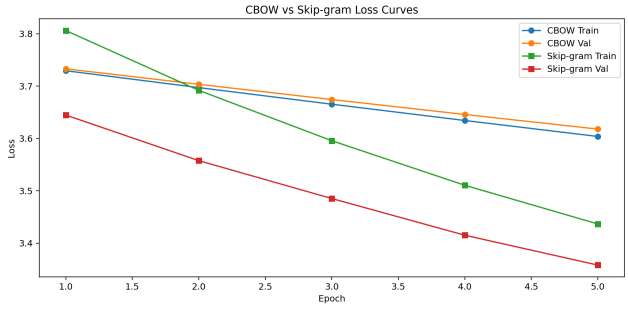

Fig. 1. CBOW and Skip-gram training and validation loss curves.

B. Ablation Study

Ablation findings are reported in Table II. Increasing context window from 1 to 4 reduced best validation loss from 3.8840 to 3.5886 in this setup. Increasing embedding dimension from 50 to 200 also improved loss.

TABLE II
Skip-gram Ablation Results

Setting	Value	Best Val Loss
Window size	1	3.8840
Window size	2	3.6431
Window size	4	3.5886
Embedding dim	50	3.7982
Embedding dim	100	3.6431
Embedding dim	200	3.5895

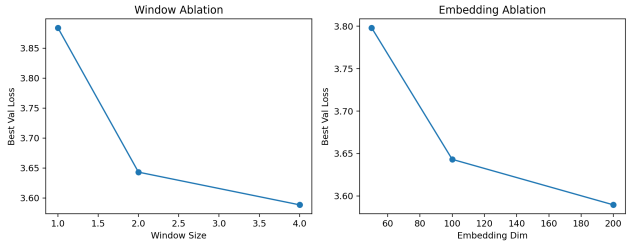

Fig. 2. Validation-loss trends for window-size and embedding-size ablations.

C. Qualitative Analysis

Embedding projections show grouping between relational terms such as king, queen, man, woman, prince, and princess, indicating that the model captures meaningful structure even in a compact corpus.

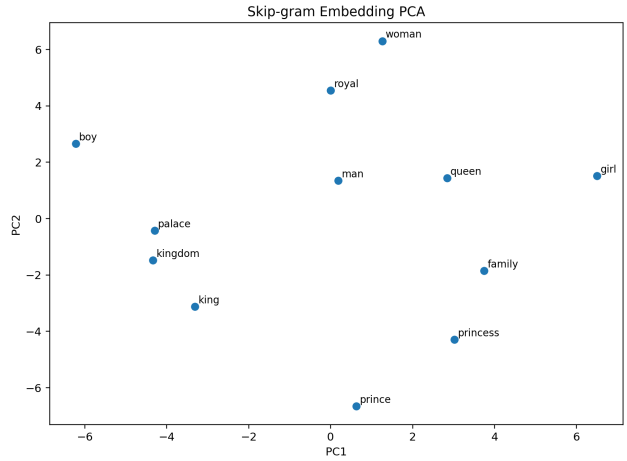

Fig. 3. Two-dimensional PCA projection of selected Skip-gram embeddings.

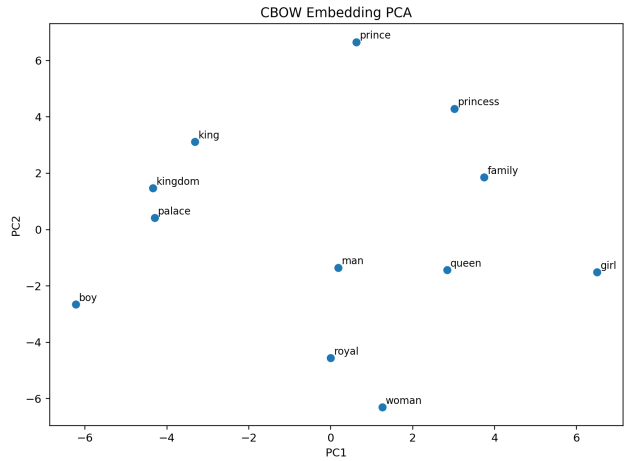

Fig. 4. Two-dimensional PCA projection of selected CBOW embeddings.

VI. Reproducibility and Artifacts

All experiments are reproducible through scripted runs that generate logs, tables, and figures. For each run, checkpoints and configuration files are stored, and the environment details are logged (Python, PyTorch, and package versions). This makes result tracing and paper writing systematic.

VII. Limitations

This study is conducted on a compact synthetic corpus with full-softmax training. As a result, semantic coverage is limited compared with large web-scale corpora. The implementation does not include hierarchical softmax or negative sampling, which are central to high-throughput Word2Vec training at scale [6], [8].

VIII. Conclusion

This paper presented a reproducible PyTorch implementation of CBOW and Skip-gram with a complete artifact-generation workflow for research reporting. Skip-gram consistently performed better than CBOW in the current setup. Ablation studies confirmed the importance of context window and embedding dimensionality. Future work includes larger corpora, negative sampling, hierarchical softmax, and broader intrinsic and extrinsic evaluation.

References

[1] Y. Bengio, R. Ducharme, and P. Vincent, "A neural probabilistic language model," Journal of Machine Learning Research, vol. 3, pp. 1137-1155, 2003.

[2] R. Collobert and J. Weston, "A Unified Architecture for Natural Language Processing: Deep Neural Networks with Multitask Learning," in Proc. ICML, 2008.

[3] R. Collobert et al., "Natural Language Processing (Almost) from Scratch," Journal of Machine Learning Research, vol. 12, pp. 2493-2537, 2011.

[4] G. E. Hinton, J. L. McClelland, and D. E. Rumelhart, "Distributed representations," in Parallel Distributed Processing: Explorations in the Microstructure of Cognition, MIT Press, 1986.

[5] T. Mikolov, W. T. Yih, and G. Zweig, "Linguistic Regularities in Continuous Space Word Representations," in Proc. NAACL-HLT, 2013.

[6] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed Representations of Words and Phrases and their Compositionality," in Proc. NIPS, 2013.

[7] A. Mnih and Y. W. Teh, "A fast and simple algorithm for training neural probabilistic language models," in Proc. ICML, 2012.

[8] F. Morin and Y. Bengio, "Hierarchical Probabilistic Neural Network Language Model," in Proc. AISTATS, 2005.

[9] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning internal representations by backpropagating errors," Nature, vol. 323, pp. 533-536, 1986.

[10] H. Schwenk, "Continuous space language models," Computer Speech and Language, vol. 21, 2007.